SafeTail 2.0

Codebase Analysis & Research-Paper Traceability Report

Function inventory · call graph · function-wise explanation · implementation gap analysis against the SafeTail camera-ready paper, the Heterogeneous Edge Devices draft, and the BTP report

Prepared for Krishna Shukla | Project: IP_Arani | 19 August 2026

Table of Contents

1. Source Documents and Naming Convention

Three documents describe this system. Throughout this report they are referred to by the short tags below, because their content overlaps heavily and precision matters when attributing a design decision.

Tag	Document	Pages	Role
ST	SafeTail_Camera_Ready.pdf — *SafeTail: Tail Latency Optimization in Edge Service Scheduling via Redundancy Management*	14	Published camera-ready. Homogeneous servers, user-centric, target-latency reward.
HED	Heterogeneous_Edge_Devices____Tail_Latency.pdf	6	Unfinished draft. Central controller, heterogeneous devices, M/M/1 queueing, sigmoid multi-select output.
BTP	SafeTail-2.0-main/documentation.pdf — *Optimising Tail Latency of Edge Applications*	28	B.Tech project report, Apr 2026. Documents **this** codebase directly (§6.3 links the repo).

The BTP report is the authoritative specification for the code: it is the only one of the three that explicitly claims to describe this repository. ST and HED are its intellectual parents. Where the code and the BTP report disagree, that is a defect; where the code and ST/HED disagree, that is usually a deliberate design evolution.

2. Repository Layout

Path	Contents
`src/*.py`	8 modules, 3,222 lines total — the entire runtime system.
`src/server{1-5}_regressor/`	15 predictor wrapper classes (3 task types × 5 servers).
`dataset/server{1-5}.csv`	Per-server hardware + execution traces, indexed by workload *combination* string.
`dataset/propagation_delays.pkl`	Pickled per-server propagation-delay sample pools.
`models/server{1-5}/*.pkl`	15 pre-trained regression models (see §9 — these are **not** MLPs).
`results/`	safetail_training_logs/, baselines/ (9 runs), deadline_manipulation/ (80% and 150% deadline scaling).
`plotting/plot_baselines.ipynb`	Comparison plots across baseline modes.
`testing/`	regressor_test.ipynb, server_test.ipynb, test_agent.py.
`run_all_baselines.sh`	Sweeps BASELINE_MODE across all 9 heuristic configurations.
`docs/`	BTP_Poster_Final.pdf, reference_material.pdf, safetail1.0.pdf.

3. Runtime Pipeline — How a Request Flows

The system is a three-process-in-one-process simulation: a synthetic traffic generator speaks TCP to a receiver, which hands chunks to a controller, which schedules onto simulated servers. Understanding this loop makes every function's role obvious.

#	Call	What happens
1	`main.main()`	Builds Controller, Receiver, SenderBursts. Calls controller.run().
2	`controller.run()`	Starts receiver.run_async() on a daemon thread; sleeps 1 s to let the socket bind.
3	`sender.run()`	Generates 500,000 Request objects via request_factory, batches them into chunks of 5, and dispatches them in random-sized bursts (2–4 chunks) over TCP with random inter-burst sleeps.
4	`receiver._process_connection()`	Length-prefixed recv → np.load(allow_pickle) → stamps arrival_time → controller.send_to_server(arr).
5	`controller.send_to_server()`	One chunk = one unit of work. Iterates requests, calls process_step() per request under a lock. After chunks_per_episode (3) chunks, calls finalize_episode().
6	`controller.process_step()`	Per request: poll server availability → estimate delay on *all* servers → populate request state from CSVs → pick action (DQN or baseline) → schedule on chosen servers → compute step reward → log.
7	`controller.finalize_episode()`	Computes episodic reward, writes it back over *every* step experience, stores to replay memory, triggers agent.experience_replay(), plots every 20 episodes.

Key structural observation

A **step** in the code is one *request*, and an **episode** is 3 chunks × 5 requests = 15 requests. Training therefore happens once per 15 requests, and every one of those 15 experiences is stored with the *same* episodic reward value, overwriting the individual step rewards that were computed. This is a meaningful deviation from standard DQN credit assignment and is discussed in §9.

4. Function Inventory by File

Line numbers refer to the current state of the repository. Private helpers are prefixed with an underscore by convention.

4.1 constants.py — configuration surface (no functions)

Pure configuration module, 69 lines, imported by every other module. Notable values:

Constant	Value	Meaning
`beta`	`5`	Number of edge servers.
`nA`	`2**beta - 1 = 31`	Action space: all non-empty server subsets.
`nS`	`1*beta + 1 = 6`	Declared state size — **unused**; the model takes variable-length input instead.
`total_no_request`	`500000`	Total synthetic requests.
`chunk_size`	`5`	Requests per TCP chunk.
`episode_size`	`4`	Declared — but Controller uses chunks_per_episode=3.
`ORIGINAL_DEADLINES`	`[[100,400],[30,200]]`	[D1,D2] in ms for 's' and for 'd'/'p'.
`DEADLINE_SCALE`	`1`	Set to 0.8 / 1.5 for the deadline_manipulation experiments.
`learning_rate`	`1e-6`	Adam LR; decays ×0.999 per replay to floor 1e-5.
`gamma_decay`	`0.002`	Epsilon decrement per replay (misleading name).
`epsilon_min`	`0.1`	Exploration floor.
`discount_rate`	`0.9`	γ for both Bellman target and episodic discounting.
`batch_size`	`128`	Replay minibatch.
`max_load`	`5`	Declared — **unused**; servers.py hardcodes 4 instead.
`BASELINE_MODE`	`"safetail"`	Switch: safetail | minload_x | minprop_x | rand_x.

4.2 main.py — 201 lines

Line	Function	Purpose
35	`print_constants(module)`	Reflectively dumps all non-callable module attributes at startup for run reproducibility.
56	`request_factory(i)`	Builds one Request: random type from {s,p,d}, message_size=1024, bandwidth=20, zero load vector, type-matched deadline. Four nested fallback levels so it can never crash the sender.
155	`main()`	Wires Controller → Receiver → SenderBursts, runs the sender (blocking), then waits on controller.training_done before stopping the receiver.

4.3 sender_bursts.py — 153 lines

Line	Function	Purpose
15	`_send_length_prefixed()`	Writes an 8-byte big-endian length header, then the payload.
19	`_make_payload_from_array()`	np.save of an object array into a BytesIO buffer.
26	`SenderBursts.__init__()`	Pre-generates all requests via request_factory, computes total_chunks.
79	`_sender_worker()`	One TCP connection: connect, send, await 1-byte ACK, return (success, duration, exception).
98	`run()`	Burst loop: random burst size in [min_burst,max_burst], one thread per chunk, join with timeout, then sleep a random interval before the next burst. Returns dispatch statistics.
152	`stop()`	Sets the stop event.

4.4 receiver.py — 340 lines

Line	Function	Purpose
19	`Receiver.__init__()`	Config + mutual backlink: sets controller.receiver_queue = self.
45	`_recv_all(conn, n)`	Loops recv until exactly n bytes are read or the peer closes.
54	`_visualize_queue()`	ASCII progress bar of pending-connection depth.
61	`_extract_request_id(req)`	Defensive ID extraction: tries ~6 attribute names, __dict__, getter methods, to_dict variants, then a dir() scan. Purely diagnostic.
130	`_process_connection()`	Per-connection loop: header → payload → np.load → persist .npy → stamp arrival_time → controller.send_to_server() → ACK.
245	`run()`	Socket bind with 10× retry on EADDRINUSE, accept-window batching into a pending deque capped at MAX_QUEUE, then serial processing.
331	`run_async()`	Launches run() on a daemon thread.
338	`stop()`	Sets the stop event.

Note: the receiver is where the only real queue in the system lives — the pending deque. It is a connection backlog, not a modelled M/M/1 service queue (see §8.2).

4.5 user.py — 320 lines

Line	Function	Purpose
15	`_get_server_df(idx)`	Module-level CSV cache; loads server{idx+1}.csv once, indexes by 'Combination' for O(1) lookup.
44	`_to_float_array(val)`	Parses list / ndarray / stringified-list CSV cells into float arrays.
73	`Request.__init__()`	Fields: request_id, process_id, combination, deadline, arrival_time, queue_waiting_time, message_size, bandwidth, load, total_processing_delay, step_reward_list, plus server_dicts[6] and server_np[5].
101	`fill_server_dict()`	Stores the rich, semantic per-server snapshot (20 named fields incl. cpu_model, gpu_model, clocks).
159	`fill_server_np()`	Flat numeric projection with a documented fixed ordering — 9 scalars then 6 variable-length arrays concatenated.
232	`populate_request_from_csv()`	Looks up the row for a combination string and calls both fill_ methods.
318	`__repr__()`	Debug string.

4.6 servers.py — 272 lines

Line	Function	Purpose
17	`Server.__init__()`	Loads propagation_delays.pkl and server{i}.csv, initialises active_requests, loads predictors.
43	`_load_predictors_from_regressor_folder()`	Dynamically imports DetectPredictor / SpeechPredictor / PredictPredictor from server{i}_regressor/. Silently sets None on any failure.
81	`print_active_requests()`	Debug dump of in-flight requests.
90	`_get_propogation_delay()`	random.choice from this server's empirical delay pool.
93	`_get_tramission_delay()`	random.choice from a **hardcoded** list [18.5,19.2,20,21.5,22]/1000 s — ignores message_size and bandwidth entirely.
96	`_collect_visible_types()`	First letter of each active request's combination, ordered by start time.
108	`_predict_using_letter()`	Runs the matching predictor; falls back to the CSV's 'Total Processing Time (sec)' column, then to 0.0.
140	`_choose_first_letter_for_regressor()`	Builds the contention string: new request's letter + reversed active letters.
157	`compute_request_time()`	**Pure estimator** — returns (total, combined_str, computation, propagation, transmission) without scheduling anything.
191	`schedule_request()`	Capacity check against MAX_CONCURRENT_REQUESTS=4, then appends to active_requests with start/finish times.
233	`update_active_requests()`	Retires requests whose finish_time has passed.
251	`check_server_availability()`	Returns current depth, or -1 if full.
261	`time_until_next_free()`	Earliest finish_time minus now. **Never called.**

4.7 agent.py — 712 lines

Line	Function	Purpose
20	`get_subsets(fullset)`	Bitmask enumeration of all 2^n–1 non-empty subsets → the action→servers lookup table.
37	`request_to_state_array()`	Flattens a Request into a 1-D float vector by walking vars(): skips dicts, coerces scalars/lists/ragged arrays, maps None/NaN/±inf to –1.0. **This is the state encoder's input contract.**
118	`DQNAgent.__init__()`	Builds the model, replay deque (maxlen 2500), and ~12 metric arrays.
168	`decay_learning_rate()`	Multiplicative LR decay to a floor.
176	`dump_replay_batch()`	Debug dump of a minibatch (off by default).
204	`timed_predict()`	Forward pass with wall-clock timing → prediction_times.
214	`build_model()`	Encoder (Dense64→Drop→Dense32→Drop→GlobalAvgPool→Dense24) + DQN head (Dense48→BN→Drop→Dense64→BN→Drop→Dense31 linear). MSE + Adam(clipnorm=1.0). End-to-end trainable.
260	`store()`	Appends (raw_state, action, reward, raw_next_state) — stores *unencoded* states so gradients reach the encoder.
271	`get_action()`	ε-greedy: random action index, or argmax over Q-values. Returns (server_subset, action_index) and logs access rate.
298	`experience_replay()`	Samples the batch, pad_sequences to uniform length, two predict() passes, Bellman target r+γ·max Q', clip to [–10,10], fit with validation_split=0.2, decay ε and LR.
363	`get_min_delay()`	Minimum estimated delay across a server subset.
372	`reward()`	**Explicitly deprecated** in its own docstring; superseded by the controller's step/episodic rewards.
389	`plot_training_loss()`	Train/validation loss subplot pair.
436	`plot_all_metrics()`	3×3 dashboard: loss, epsilon, explore/exploit, reward, latency, access rate, deviation.
619	`_draw_testing_boundary()`	Vertical marker at the train→test transition.
636	`save_metrics_summary()`	**Entire body commented out** — a no-op `pass`.

4.8 controller.py — 1,155 lines (the orchestrator)

Line	Function	Purpose
15	`Controller.__init__()`	Builds 5 Servers and the DQNAgent, opens 5 CSV log files, sets episode counters, optionally loads a saved model for the testing phase.
131	`log_request_access_rate_with_type()`	Appends to request_wise_access_log.csv.
142	`log_latency_to_csv()`	Appends the 5-way latency breakdown per request.
173	`_safe_read_csv()`	Guarded pd.read_csv returning an empty frame on failure.
182	`load_existing_plot_history()`	Rehydrates rewards / access / latency arrays from a previous run and records the testing-start indices.
227	`find_free_servers()`	Vector of per-server availability (–1 = full).
234	`_select_minload_servers(x)`	Baseline: x servers with fewest active requests.
240	`_select_minprop_servers(x)`	Baseline: x servers with lowest propagation delay.
246	`_select_rand_servers(x)`	Baseline: x random servers.
254	`get_queue_lengths()`	Per-server len(active_requests). **Never called.**
258	`dispatch_to_agent()`	Thin passthrough to agent.get_action(). **Never called** — process_step calls the agent directly.
261	`assign_request()`	**Never called, and broken** — unpacks 3 values from schedule_request(), which returns 7.
285	`compute_step_reward()`	Two jobs in one function: (a) when action_subset is given, computes P(T) and accumulates type counters; (b) always, returns per-server reward log((1–cm)(1–cu)(1–gm)(1–gu)+1).
456	`compute_episodic_reward()`	$\Sigma\gamma^{j}R_{step,i}$ + ω – W■/denominator, clipped to [–10,10].
532	`process_step()`	The core per-request routine — 228 lines. See §6.4.
760	`finalize_episode()`	Episodic reward → overwrite all step experiences → store → experience_replay → plots → reset counters → set training_done when episodes exhausted.
890	`_save_and_enter_testing()`	Saves .keras model, records per-metric testing-start indices, sets ε=0, clears replay memory.
931	`generate_testing_plots()`	**Empty stub** — `pass`.
934	`generate_plots()`	Periodic (every 20 episodes) dashboard + loss plot.
967	`generate_final_plots()`	End-of-training high-res plots.
1006	`export_training_data()`	**Entire body commented out** — no-op.
1055	`save_checkpoint()`	Model + metrics CSV checkpoint. **Call site commented out**.
1079	`send_to_server(chunk)`	Chunk loop under a lock; increments chunk counter; finalizes the episode when chunks_per_episode is reached.
1151	`run()`	Starts the receiver thread and waits 1 s for bind.

4.9 server{1-5}_regressor/*.py — 15 near-identical wrappers

Each file defines one class — `DetectPredictor`, `SpeechPredictor`, or `PredictPredictor` — with an identical four-method shape:

Function	Purpose
`__init__()`	Loads models/server{i}/{task}_regressor_model.pkl (a dict with keys model, scaler, feature_columns, model_name) and the matching CSV.
`_find_row(combination)`	Case-insensitive lookup of the row whose 'Combination' matches.
`_build_features(row)`	Constructs 11 features: num_speech, num_detect, num_predict, total_ops, position, is_first, is_last, peak_ram, peak_gpu, peak_gpu_memory, total_processing_time.
`predict_from_combination()`	_find_row → _build_features → scaler.transform → model.predict → float.

Critical observation. `_build_features` reads `total_processing_time` straight out of the same CSV row it is trying to predict a component of, and feeds it in as an input feature. The 'prediction' is therefore partly a lookup of ground truth, not a forecast from independent system state. Live CPU/GPU utilisation of the *target* server at request time is never consulted — only the static trace row keyed by the contention string.

5. Call Graph — Who Calls What

Arrows read *caller* → *callee*. Cross-module calls are the interesting ones; trivial logging helpers are omitted.

```
main()
 |-> Controller.__init__()
  |     |-> servers.Server.__init__() x5
  |      |     `-> Server._load_predictors_from_regressor_folder()
  |      |           `-> {Detect,Speech,Predict}Predictor.__init__()
  |     `-> agent.DQNAgent.__init__()
  |           |-> agent.get_subsets()
  |           `-> DQNAgent.build_model()
 |-> Receiver.__init__()          [sets controller.receiver_queue = self]
 |-> Controller.run()
  |     `-> Receiver.run_async() --> Receiver.run()  [daemon thread]
  |           `-> Receiver._process_connection()
  |                 |-> Receiver._recv_all()
  |                 |-> Receiver._extract_request_id()
  |                 `-> Controller.send_to_server()        <== THREAD BOUNDARY
 `-> SenderBursts.run()
       `-> SenderBursts._sender_worker()  [one thread per chunk]

Controller.send_to_server(chunk)
 |-> Controller.process_step(request)          [per request, under self.lock]
  |     |-> Controller.find_free_servers()
  |      |     `-> Server.check_server_availability()
  |      |           `-> Server.update_active_requests()
  |     |-> Server.compute_request_time()        [x5, all servers]
  |      |     |-> Server._get_propogation_delay()
  |      |     |-> Server._get_tramission_delay()
  |      |     |-> Server._choose_first_letter_for_regressor()
  |      |      |     `-> Server._collect_visible_types()
  |      |     `-> Server._predict_using_letter()
  |      |           `-> Predictor.predict_from_combination()
  |      |                 |-> Predictor._find_row()
  |      |                 `-> Predictor._build_features()
  |     |-> Request.populate_request_from_csv()  [x5]
  |      |     |-> user._get_server_df()   [cached]
  |      |     |-> user._to_float_array()
  |      |     |-> Request.fill_server_dict()
  |      |     `-> Request.fill_server_np()
  |     |-> Controller.compute_step_reward(request)          [pre-action, all servers]
  |     |-> ACTION SELECTION ■ one of:
  |      |     |-> DQNAgent.get_action()            [BASELINE_MODE == "safetail"]
```

```
  |      |      |     |-> agent.request_to_state_array()
  |      |      |     `-> DQNAgent.timed_predict()
  |      |     |-> Controller._select_minload_servers()
  |      |     |-> Controller._select_minprop_servers()
  |      |     `-> Controller._select_rand_servers()
  |     |-> Server.schedule_request()             [per selected server]
  |      |     |-> Server.update_active_requests()
  |      |     `-> Server.compute_request_time()   [recomputed - see §9]
  |     |-> Controller.compute_step_reward(request, action_subset)   [post-action]
  |     |-> Controller.log_latency_to_csv()
  |     |-> Controller.log_request_access_rate_with_type()
  |     `-> Controller._save_and_enter_testing()  [once, when post-epsilon budget spent]
 `-> Controller.finalize_episode()               [every 3 chunks]
       |-> Controller.compute_episodic_reward()
       |-> DQNAgent.store()          [x N step experiences]
        |     `-> agent.request_to_state_array()
       |-> DQNAgent.experience_replay()
        |     `-> DQNAgent.decay_learning_rate()
       |-> Controller.generate_plots()           [every 20 episodes]
        |     |-> DQNAgent.plot_all_metrics()
        |     |-> DQNAgent.plot_training_loss()
        |     `-> DQNAgent.save_metrics_summary()   [no-op]
       `-> Controller.generate_final_plots()      [last episode]

ORPHANED (defined, never called):
   Controller.assign_request()      - and broken: 3-tuple unpack of a 7-tuple
   Controller.dispatch_to_agent()
   Controller.get_queue_lengths()
   Controller.save_checkpoint()     - call site commented out
   Controller.export_training_data()- body commented out
   Controller.generate_testing_plots() - empty stub
   Server.time_until_next_free()
   Server.print_active_requests()
   DQNAgent.reward()                - docstring marks it DEPRECATED
   DQNAgent.get_min_delay()         - only reachable from the deprecated reward()
   DQNAgent.save_metrics_summary()  - body commented out
```

6. Function-wise Explanation

6.1 Entry point and traffic generation

main.request_factory(i) is deliberately paranoid: four nested fallbacks (full Request → minimal Request → SimpleNamespace → None) because it runs inside the sender's hot loop where an exception would kill traffic generation. Note that it hardcodes `message_size=1024` and `bandwidth=20` for every request — so input-size variability, which both ST and HED treat as a first-class driver of transmission latency, does not exist in the generated workload.

SenderBursts.run() produces burstiness with `random.randint(min_burst, max_burst)` chunks per burst and `random.uniform(min_interval, max_interval)` seconds between bursts. This is uniform-random burstiness, *not* a Poisson arrival process (see §8.2).

6.2 Request and server modelling

Request.fill_server_np() is the contract that makes heterogeneity tractable. It concatenates 9 scalars (ram_usage, gpu_usage, gpu_memory, time_exec, time_process, cpu_core_used, total_ram, total_cpu_cores, total_gpu_memory) with 6 variable-length arrays (cpu_usage per core, per-script process/exec times, files per script, cpu_clock, gpu_clock). Because core counts differ across servers, these vectors have *different lengths per server* — which is precisely the problem the encoder exists to solve.

Server.compute_request_time() sums three terms: propagation (sampled from the empirical pool), transmission (sampled from a hardcoded 5-element list), and computation (regressor output). Critically it is a *pure* function — it mutates nothing — which is why process_step can safely call it on all 5 servers to build the state before committing to an action.

Server._choose_first_letter_for_regressor() encodes contention as a string. If a detection request arrives while a speech and a predict job are already running, the combined string becomes something like `"dps"`, and the regressor is asked for the row matching that exact co-execution pattern. This is a clever trace-lookup scheme, but it means contention is modelled only at the granularity of *which task types* are co-resident — not how loaded the machine actually is right now.

6.3 The DQN agent

agent.request_to_state_array() flattens the entire Request via `vars()`. This is elegant but fragile: adding any attribute to Request silently changes the state vector's length and meaning, and the ordering depends on attribute insertion order. Dicts are skipped, so `server_dicts` contributes nothing — only `server_np` matters.

DQNAgent.build_model() merges the encoder and the Q-network into one graph so gradients flow end-to-end. The encoder handles variable length via `Input(shape=(None,1))` plus `GlobalAveragePooling1D`. The docstring records the anti-overfitting changes made during development: layer widths halved, sigmoid replaced with ReLU, dropout added at 0.3/0.4, and — importantly — softmax removed from the output because it is wrong for Q-value regression. That last change is exactly where the code parts ways with ST's specified architecture (§8.1).

DQNAgent.experience_replay() pads variable-length states with −1.0 to a common length, computes standard Bellman targets, and clips them to [−10,10] to stop MSE from exploding. The `validation_split=0.2` on a replay minibatch is unusual — it holds out 20% of a randomly sampled batch purely to produce a val_loss curve for plots; it does not regularise anything meaningfully.

6.4 The controller — scheduling, reward, training

process_step(request) — the heart of the system, in eight phases:

Phase	What it does	Note
1. Availability	find_free_servers(); if all full, sleep 0.1 s and **recurse**.	Unbounded recursion — a sustained full cluster overflows the stack.
2. Estimate	compute_request_time() on all 5 servers; store into request.total_processing_delay.	This is the lookahead the agent sees.
3. Hydrate	populate_request_from_csv() per server using the returned combination string.	Fills server_dicts + server_np.
4. Pre-reward	compute_step_reward(request) with no subset — all servers.	Result is stored on the request but never used for training.
5. Act	DQN or baseline selection, per BASELINE_MODE.	Logs access rate for safetail mode.
6. Wait	queue_waiting_time = now – arrival_time.	Measured wall-clock, not an M/M/1 estimate.
7. Schedule	schedule_request() on each selected server; collect observed delays.	Recomputes delay — fresh random draws, so it differs from phase 2.
8. Post-reward	compute_step_reward(request, action_subset) → P(T), counters, mean reward, CSV logs.	Only now is P(T) accumulated.

compute_step_reward() returns, per server, `log((1–cm)(1–cu)(1–gm)(1–gu) + 1)` where the four terms are RAM, CPU-core, GPU-memory and GPU-core utilisation fractions. The product is near 1 for a fully idle server and near 0 for a saturated one, so the reward lies in [0, log 2] — it is a pure *resource-headroom* signal. Latency does not appear in it at all. The docstring advertises a different formula (`1 + log(exp(·) – 1)`) than the code implements.

compute_episodic_reward() combines three terms: the γ-discounted sum of step rewards, the satisfaction ω = ΣP(T)/expected_requests_per_episode, and a normalised average-waiting-time penalty. The waiting-time denominator is a weighted sum of D1 values scaled by per-type completion ratios — but those ratios are computed from `request_x_done` counters that are incremented in `send_to_server` for *every* request regardless of whether it met its deadline (the deadline-conditional increments in compute_step_reward are commented out). The ratio is therefore effectively 1.0 by construction.

finalize_episode() performs the credit assignment that most defines this system's learning behaviour: every stored experience receives the *episodic* reward, not its own step reward. Combined with `next_state == state` in the stored tuple, the temporal-difference target degenerates towards `r + γ·max Q(s)` on the same state — a substantial departure from textbook DQN.

7. Implementation Traceability Matrix

Every row below was verified against the source. 'Where' gives the concrete code location.

7.1 Implemented — from the SafeTail camera-ready paper (ST)

ST concept	Paper reference	Where implemented	Fidelity
Redundant allocation; take the fastest response	Eq. 2, $L_R=\min L_i$	`process_step ph.7-8; min over total_processing_delay`	Faithful
Subset action space, 2^n-1 actions	§IV, $\|A\|=2^n-1$	`agent.get_subsets(); constants.nA=31`	Faithful
ε-greedy with decay to ε_{min}	Eq. 3	`DQNAgent.get_action(); experience_replay decay`	Faithful
Capacity-aware admission; overflow refused	§II assumption (i)–(ii)	`servers.MAX_CONCURRENT_REQUESTS=4`	Faithful
n=5 servers, capacity 4	§V-D	`constants.beta=5; MAX_CONCURRENT_REQUESTS=4`	Exact match
Rand-x baseline	§V-B (ii)	`_select_rand_servers(); rand_1/2/3`	Faithful
MinProp-x baseline	§V-B (iii)	`_select_minprop_servers()`	Faithful
MinLoad-x baseline	§V-B (iv)	`_select_minload_servers()`	Faithful
Access rate metric	§V-C (i)	`access_rate = len(subset)/num_servers`	Faithful
Learned encoder for state	§IV FNN input	`build_model() encoder block`	Concept kept, architecture changed
Train/test split with saved model	§V-C	`_save_and_enter_testing(); ε=0`	Faithful

7.2 Implemented — from the Heterogeneous Edge Devices draft (HED)

HED concept	Paper reference	Where implemented	Fidelity
Central controller between users and devices	Abstract item 1	`controller.Controller`	Faithful
Static + dynamic heterogeneous server state	Abstract item 5	`Request.fill_server_dict / fill_server_np`	Faithful — all named fields present
Encoder to unify differing state dimensions	Abstract item 6	`build_model()` encoder + `pad_sequences`	Faithful in purpose
Per-application latency regressors	§IV-D	`server{1-5}_regressor/*; models/*.pkl`	Present, but not MLPs — see §9
Two-tier deadlines D1/D2 per service type	Abstract item 9	`constants.ORIGINAL_DEADLINES=[[100,400],[30,200]]`	Exact values match
Degree of satisfaction ω	§III-E	`compute_episodic_reward(); P(T)` in `compute_step_reward`	Reformulated — BTP's cleaner piecewise version
Step + episodic two-level reward	§III-E	`compute_step_reward / compute_episodic_reward`	Structure kept, formulas differ
Batch of k requests per step	§III-A	`send_to_server` iterates a chunk of 5	Partial — chunked, but decided one at a time
Three workload types (detect / segment / speech)	§IV-B	`combination ∈ {"s","d","p"}`	Faithful

7.3 Implemented — introduced by the BTP report (BTP)

These have no clean ancestor in either research paper and appear to be BTP-era additions.

BTP concept	Report reference	Where implemented
Piecewise linear P(T) satisfaction with partial credit	§3.7	`compute_step_reward lines 320–361`
Integrated end-to-end Encoder+DQN (one graph)	§4.3	`build_model()`
Deadline-scaling experiments (80% / 150%)	Ch. 5	`constants.DEADLINE_SCALE; results/deadline_manipulation/`
Controller-queue term in the latency model	§3.4	`request.queue_waiting_time; logged as queueing_delay`
Baseline sweep automation	§4.7	`BASELINE_MODE + run_all_baselines.sh`
TCP transport between generator and controller	—	`sender_bursts.py + receiver.py`

8. Not Implemented — Gap Analysis

8.1 From the SafeTail camera-ready paper (ST)

ST feature	Reference	Status in code	Impact
Target latency $\tau(\omega_t)$	Def. 4.2, Eq. 4	**Absent.** No τ is computed anywhere; no variable resembles it.	**High** — this is ST's central mechanism.
Five-case reward keyed to τ and $\lvert E_k \rvert$	Def. 4.3, Eq. 5	**Absent.** Replaced by a resource-headroom product + P(T).	**High** — the asymmetric over/under-target penalty is gone.
Target-vector construction, sum-to-1	Eq. 6	**Absent.** Standard Bellman targets used instead.	High
5 hidden layers, softmax, categorical cross-entropy	§IV	**Changed.** 2 DQN hidden layers, linear output, MSE.	Medium — different model family (value vs. policy).
Oracle (optimal) baseline	§V-B (i)	**Absent.** No 'oracle' string in the repo.	**High** — no optimality gap can be computed.
DRL-Linear baseline	§V-B (v)	**Absent.**	Medium
TLORA baseline	§V-B (vi)	**Absent.** No Weibull modelling anywhere.	Medium
Homogeneous-server assumption	§II	**Deliberately inverted.** Per-server CSVs and models.	Intended — this is the point of 2.0.
Local fallback service on the user device	§II	**Absent.** A full cluster causes recursion, not fallback.	Medium
Latency-deviation-from-τ metric	§V-C (ii)	**Repurposed.** Deviation is measured against median_computation_delay=0.05, a fixed constant.	Medium

8.2 From the Heterogeneous Edge Devices draft (HED)

HED feature	Reference	Status in code	Impact
Sigmoid multi-label output over n·k	Abstract item 8	**Absent.** Single discrete action over 31 subsets; linear output + argmax.	**High** — HED's signature mechanism, dropped.
Argmax fallback when sigmoid selects nothing	Abstract item 8	**Absent** (moot without sigmoid).	—
M/M/1 closed form $W=\lambda/(\mu(\mu-\lambda))$	§III-D	**Absent.** No λ or μ variable exists. queue_waiting_time is measured wall-clock time.	**High** — claimed in README and BTP §3.5.
Poisson arrivals	§II-C assumption 4	**Absent.** random.randint burst sizes, random.uniform gaps.	Medium — invalidates the M/M/1 premise anyway.
Exponential service times	§II-C assumption 5	**Absent.** Regressor output on empirical traces.	Low — arguably more realistic.
Job-length-proportional priority	Abstract item 9	**Absent.** No job-size term in either reward; no priority ordering.	**High** — large jobs get no protection.
/proc-based waiting-time measurement	Unlabelled §	**Absent from src/.** Presumably offline trace tooling only.	Low
FLOPs and signal strength as RL inputs	Abstract item 7	**Absent.** Request has message_size and bandwidth only, both hardcoded constants.	Medium
Batch decision over k requests jointly	§III-A	**Partial.** Chunks of 5 arrive together but are decided sequentially and independently.	Medium
Controller M/M/1 queue	§I	**Absent.** Receiver has a connection backlog deque, not a service queue.	Medium

8.3 From the BTP report itself (BTP)

These are the most serious gaps, because the BTP report claims to document *this* code.

BTP claim	Reference	Status in code	Impact
MLP regressors for computation latency	§2.6, §4.6, Abstract	**Contradicted.** The 15 shipped .pkl models are LinearRegression, RandomForest and GradientBoosting — no MLPRegressor in any file.	**High** — a factual error in the report.
Mask R-CNN and Whisper as the workloads	§2.6	**Unverifiable in code.** Only the letters s/d/p and trace CSVs exist.	Low
M/M/1 at both controller and each server	§3.5	**Absent.** Neither queue is modelled analytically.	**High** — stated as a contribution.
W_j = min over selected servers	§3.5	**Absent.** There is no per-server W to minimise over.	High
$\omega = (1/N)\Sigma P(T_j)$	§3.7	**Implemented**, with N = total_no_request/no_of_episodes rather than the actual count of requests in the episode.	Low — constant scaling.
Step = one chunk of k requests	§3.6	**Contradicted.** current_step increments once per *request*, not per chunk.	Low — terminology drift.
Episode = fixed number of steps	§3.6	**Implemented** as 3 chunks; constants.episode_size=4 is unused and inconsistent.	Low
Load balancing as an explicit objective	§3.8 goal 3	**Implicit only** — the headroom reward favours idle servers, but no balance term or variance penalty exists.	Medium
Tail latency minimised by redundancy	§3.8 goal 1	**Indirect.** Redundancy exists, but nothing in the reward references a percentile or a latency target.	**High** — the headline objective is not encoded in the objective function.
Deadline satisfaction maximised	§3.8 goal 2	**Implemented** via ω in the episodic reward.	—

9. Discrepancies, Bugs and Dead Code

Findings from reading the source, ordered by severity. Each is reproducible from the line numbers given in §4.

Severity: high

#	Finding	Location
1	**No MLP regressors exist.** All three documents claim MLPs. Byte inspection of the 15 .pkl files shows LinearRegression (6), RandomForest/DecisionTree (8) and GradientBoosting (1). The wrapper even defaults model_name to "Linear Regression".	`models/server*/*.pkl; detect_predictor.py:35`
2	**Target-leaking feature.** _build_features passes total_processing_time from the same CSV row into the model predicting processing time.	`*_predictor.py:77`
3	**next_state == state.** Every experience stores the identical object as both current and next state, so the Bellman bootstrap adds $\gamma \cdot \max Q(s)$ to itself.	`controller.py:724–729`
4	**Step rewards are discarded.** All experiences in an episode are stored with the episodic reward, overwriting per-step credit.	`controller.py:797–803`
5	**Unbounded recursion under saturation.** process_step calls itself after a 0.1 s sleep when all servers are full, with no depth limit.	`controller.py:546–550`

Severity: medium

#	Finding	Location
6	**Delay is computed twice with different random draws.** Phase 2 estimates via compute_request_time; phase 7 calls schedule_request which calls it again, drawing fresh propagation and transmission samples. The state the agent saw is not the outcome it is graded on.	`controller.py:564 vs 659`
7	**request_x_done counters are unconditional.** The deadline-conditional increments are commented out in compute_step_reward, while send_to_server increments unconditionally — making done/total ≡ 1.	`controller.py:366–377, 1114–1119`
8	**Docstring/code mismatch in the step reward.** Docstring says 1+log(exp(·)–1); code computes log(product+1).	`controller.py:290 vs 434`
9	**Transmission delay ignores message size and bandwidth.** Sampled from a hardcoded 5-element list, so the I/O terms in ST Eq. 1 are effectively constant.	`servers.py:93–94`
10	**Deadline naming collision.** HED labels the 100/400 pair 'segmentation'; the code and BTP Table 3.1 label it 'speech'. The letters s/d/p are used inconsistently across documents.	`constants.py:41–48`
11	**request.combination is overwritten with the contention string.** After scheduling, combination becomes e.g. "dps", so downstream `combination[0]` type checks read the first letter of a mutated string.	`servers.py:215; controller.py:335`

Severity: low — dead or vestigial code

#	Finding	Location
12	assign_request() unpacks 3 values from a 7-tuple — would raise if ever called.	`controller.py:267`
13	Six controller methods and three agent methods are never called (full list in §5).	`§5 orphan list`
14	constants.nS, episode_size and max_load are declared but unused; the real values are hardcoded elsewhere.	`constants.py:22,27; servers.py:13`
15	save_metrics_summary and export_training_data have fully commented-out bodies.	`agent.py:636; controller.py:1006`
16	np.load(allow_pickle=True) on socket data — arbitrary-code-execution risk if the port is ever exposed beyond localhost.	`receiver.py:167`

10. Summary Scorecard

Document	Core mechanisms	Implemented	Partial	Absent
SafeTail (ST)	11 tracked	6	2	3 (τ, Eq. 5 reward, 3 baselines)
Heterogeneous (HED)	10 tracked	4	2	4 (sigmoid, M/M/1, Poisson, job priority)
BTP report	10 tracked	4	2	4 (MLP, M/M/1 ×2, tail objective)

What the codebase actually is

Stripped of the papers' vocabulary, the running system is: **a heterogeneity-aware, redundancy-capable DQN scheduler whose objective function is resource headroom plus deadline satisfaction.** It inherits its *structure* from HED (central controller, per-device heterogeneous state, learned encoder, two-level reward), its *action space and baselines* from ST (subset enumeration, ε-greedy, capacity admission, Rand/MinLoad/MinProp), and its *satisfaction function* from the BTP report's cleaner reformulation.

The single most consequential gap is that **tail latency — the stated objective of all three documents — never appears in the reward signal.** ST encoded it through the target latency τ and the asymmetric penalty for exceeding it; that mechanism was not carried over. What replaced it optimises for idle resources and deadline hits, which correlates with good tail behaviour but does not target it. Any percentile improvement observed in results is therefore an emergent side-effect of redundancy, not a directly optimised quantity.

The second most consequential gap is the **missing Oracle baseline**. Without it, the optimality-gap analysis that made ST's evaluation persuasive (Table V: SafeTail within 8–30% of optimal while using 1–3 of 5 servers) cannot be reproduced here.

Suggested priorities if closing the gaps

Priority	Action	Why
1	Retrain or relabel the regressors, and drop the leaking total_processing_time feature.	Correctness of every latency number downstream; also fixes a factual claim in the report.
2	Add the Oracle baseline (schedule to all 5, record the min).	Cheap to implement; unlocks the optimality-gap table.
3	Introduce τ and a latency-referenced reward term.	Restores the actual tail-latency objective.
4	Fix next_state to the post-action state, and store per-step rewards.	Makes the DQN a real DQN.
5	Implement M/M/1 waiting time, or remove the claim from README and BTP §3.5.	Align documentation with reality either way.
6	Reuse the phase-2 delay draw in phase 7.	Ensures the agent is graded on the state it observed.

Prepared by reading every line of src/ (3,222 lines across 8 modules), the 15 regressor wrappers, the 15 serialised models, and the three source documents. All line numbers, model types and formula discrepancies were verified directly against the repository rather than inferred.